# Artificial Neural Network

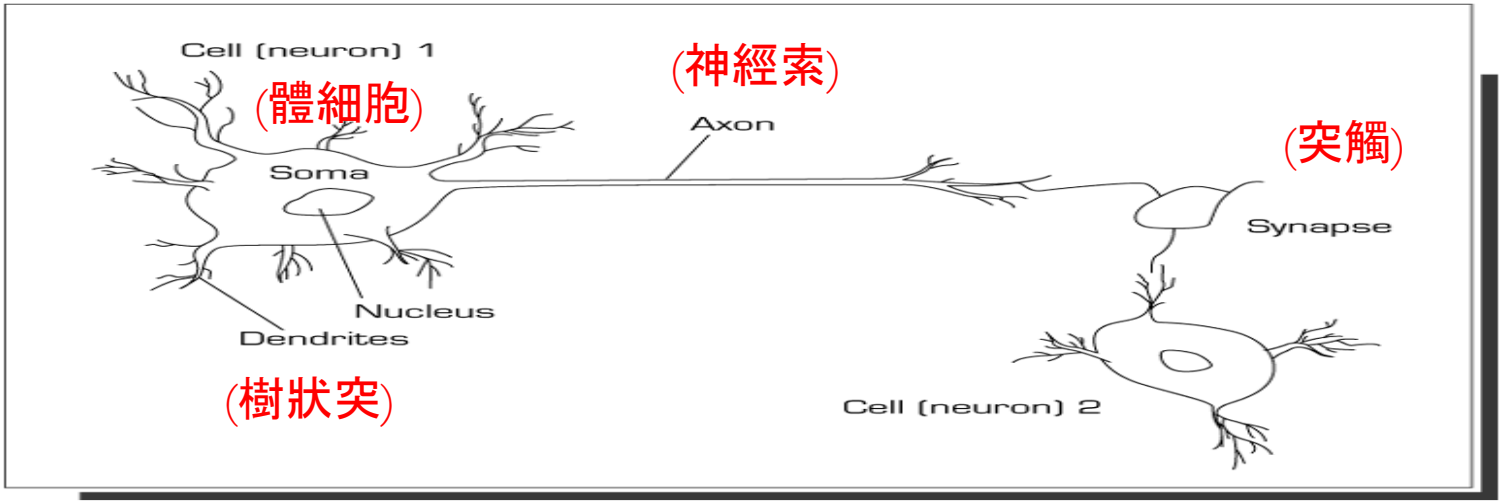
# Artificial Neural Network, Why?

- Mimic how human brains function
  - Human brain is superior to digital computers in many ways
    - A baby can recognize many objects
  - Computers are good at arithmetic
- Human brains are
  - Robust and fault-tolerant
  - Flexible and adaptive (learning)
  - Can deal with fuzzy, probabilistic, noisy or inconsistent information
  - Small, compact & consumes low power

# Human Brain

- 50 to 150 billion neurons in brain
- Neurons grouped into networks
  - Axons send outputs to cells
  - Received by dendrites, across synapses

**Figure 12.4** Portion of a Network: Two Interconnected Biological Cells



# Functions of Neurons

- Transmission of Signal
  - Complicated chemical process
  - Transmitter substances are released from the sending site
  - Raise or lower electrical potential inside the body of receiving cell
  - If the potential reaches a threshold, a pulse of a fixed strength and duration is sent (fired)
  - After firing, the cell has to wait a time called refractory period before it can fire again

# Math Model of A Neuron

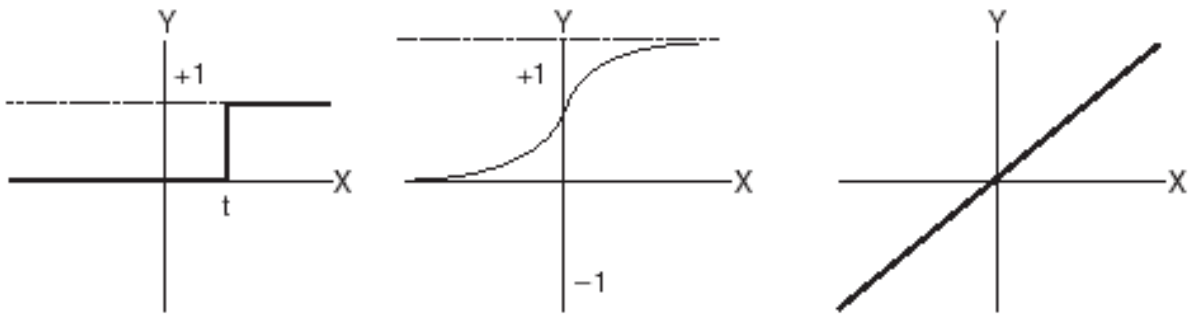
- $y_i(t+\Delta t) = g(\Sigma_j\, w_{ji} x_j(t) - \mu_i)$.
- $g(Y) = 1$ if $Y \geq 0$

  $0$ otherwise.
- $w_{ji}$: the strength of synapse connecting neuron j to neuron i
- $\mu_i$ : threshold value for neuron i
- If the weighted sum of the inputs to the neuron i exceeds the threshold $\mu_i$, the neuron fires.

# Artificial Neurons

- Artificial neurons are based on biological neurons.
- Each neuron in the network receives one or more inputs.
- An activation function is applied to the inputs, which determines the output of the neuron – the activation level.

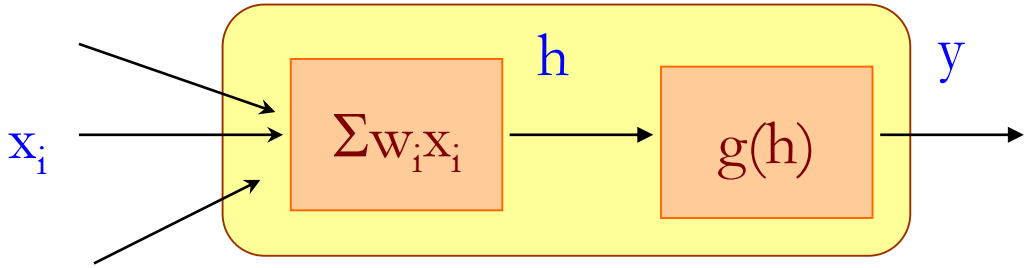
- The charts on the right show three typical activation functions g(·).




(a) Step funtion (b) Sigmoid funtion (c) Linear funtion

# Perceptrons

- A perceptron is a single neuron that classifies a set of inputs into one of two categories.
- If the inputs are in the form of a grid, a perceptron can be used to recognize visual images of shapes.
- The perceptron usually uses a step function, which returns 1 if the weighted sum of inputs exceeds a threshold, and –1 otherwise.



# Training for Perceptrons

- The perceptron is trained as follows:
  - First, inputs are given random weights (usually between –0.5 and 0.5).
  - An item of training data is presented. If the perceptron mis-classifies it, the weights are modified according to the following:

    $w_i' \leftarrow w_i + a \cdot x_i \cdot e$ ($e = d - y$, $d$ : desired output)
  - e is the size of the error
  - a is the learning rate, between 0 and 1.
- $h_i' = w_i' x_i = (w_i + a x_i e) x_i = w_i x_i + a x_i^2 e = h_i + a x_i^2 e$

  if $e < 0$ : $d < y$, y too big → updated weight $w_i'$

  → decrease h ($h_i' < h_i$) → decrease y (since $g(\cdot)$ monotonic))

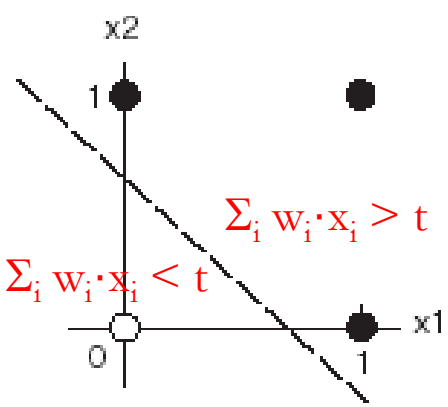
# Learning OR Function

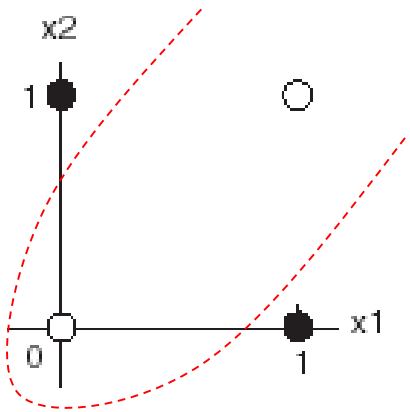
- OR
  - 0 0 → 0
  - 0 1 → 1
  - 1 0 → 1
  - 1 1 → 1
- Initial setting
  - $w_1 = -0.2$
    $w_2 = 0.4$
    $a = 0.2$

| Epoch | X1 | X2 | Expected Y | Actual Y | Error | w1 | w2 |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | −0.2 | 0.4 |
| 1 | 0 | 1 | 1 | 1 | 0 | −0.2 | 0.4 |
| 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0.4 |
| 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0.4 |
| 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0.4 |
| 2 | 0 | 1 | 1 | 1 | 0 | 0 | 0.4 |
| 2 | 1 | 0 | 1 | 0 | 1 | 0.2 | 0.4 |
| 2 | 1 | 1 | 1 | 1 | 0 | 0.2 | 0.4 |
| 3 | 0 | 0 | 0 | 0 | 0 | 0.2 | 0.4 |
| 3 | 0 | 1 | 1 | 1 | 0 | 0.2 | 0.4 |
| 3 | 1 | 0 | 1 | 1 | 0 | 0.2 | 0.4 |
| 3 | 1 | 1 | 1 | 1 | 0 | 0.2 | 0.4 |

# Limitation of A Perceptron

- Perceptrons can only classify linearly separable functions (Using step function).
- Can NOT model the problem that is not linearly separable (e.g. a simple Exclusive-OR).
  - No line can separate the black/white dots for EOR




$X = \Sigma_i w_i \cdot x_i$

$Y = +1$ if $X > t$

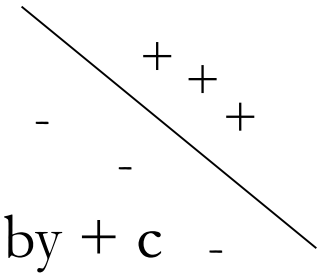
$\quad -1$ if $X < t$

$\Sigma_i w_i \cdot x_i > t \qquad +1$

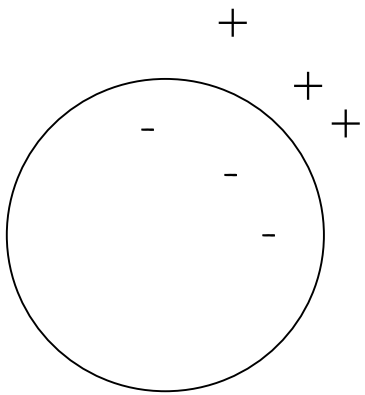
$\Sigma_i w_i \cdot x_i < t \qquad -1$

# Decision Functions for Classifiers

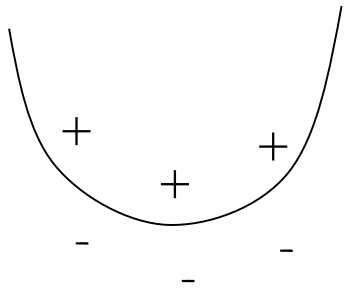
Decision boundary $f(x, y) = 0$ can separate the 2-D plane into two regions (+ or –).



$f(x,y) = ax + by + c$
$(a, b > 0)$



$f(x,y) = (x-x_0)^2 + (y-y_0)^2 - r^2$



$f(x,y) = y - ax^2 - bx - c$
$(b^2-4ac > 0)$

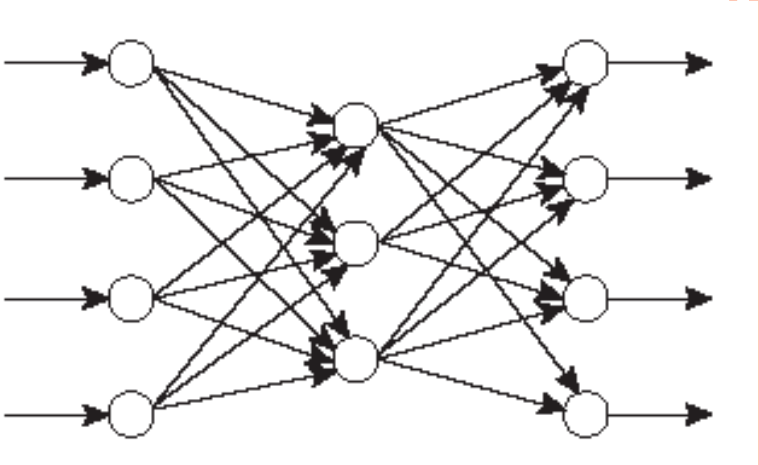
# Generalization

- $f(x, y, z) = 0$ can separate the 3D space into two regions (+ and -)
  - Decision boundary $f(x, y, z) = 0$ forms a 2D surface (e.g. a plane for linear case)
- $f(\underline{x}) = 0$ will separate the N-dimensional space into two regions (+ and -)
  - Decision boundary $f(\underline{x}) = 0$ forms a (N-1)-dimensional space
- M-ary classifier separate the space into M regions
  - $y = f(\underline{x})$ $\quad y = y_1, y_2, \ldots, y_M$
- $\underline{y} = f(\underline{x})$ arbitrary function f to be learned (not classifier)

# Multilayer Perceptrons (MLP)

- Multilayer neural networks can classify a range of functions, including nonlinearly separable ones.

  - Each input layer neuron connects to all neurons in the hidden layer.
  - The neurons in the hidden layer connect to all neurons in the output layer.



**A feed-forward network**

# Backpropagation (1)

- Multilayer neural networks learn in the same way as *perceptrons*.
- However, there are many more weights, and it is important to assign credit (or blame) correctly when changing weights.
- Backpropagation networks use the sigmoid activation function, as it is easy to differentiate:

$$\sigma(x) = \frac{1}{1+e^{-x}} \qquad \frac{d\sigma(x)}{dx} = \sigma(x)\cdot(1-\sigma(x))$$

# Backpropagation (2)

- For node j, $X_j$ is the output

  $Y_j$ is the output

  n is the number of inputs to node j

  $\theta_j$ is the threshold for j

$$X_j = \sum_{i=1}^{n} w_{ij} - \theta_j$$

$$Y_j = \frac{1}{1 + e^{-X_j}}$$

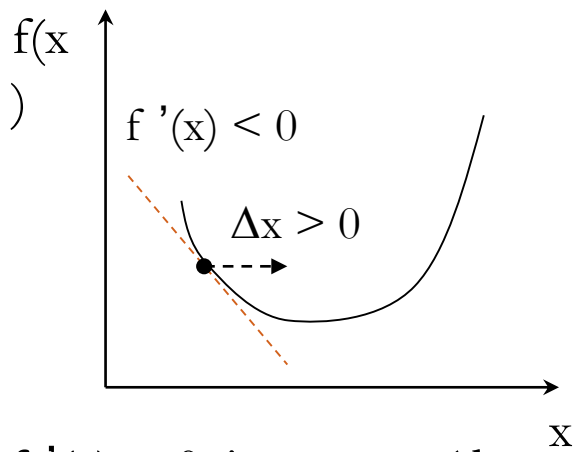
- After values are fed forward through the ne[illegible] fed back to modify the weights in order to train the network.
- For each node, we calculate an error gradient.

# Back Propagation (3)

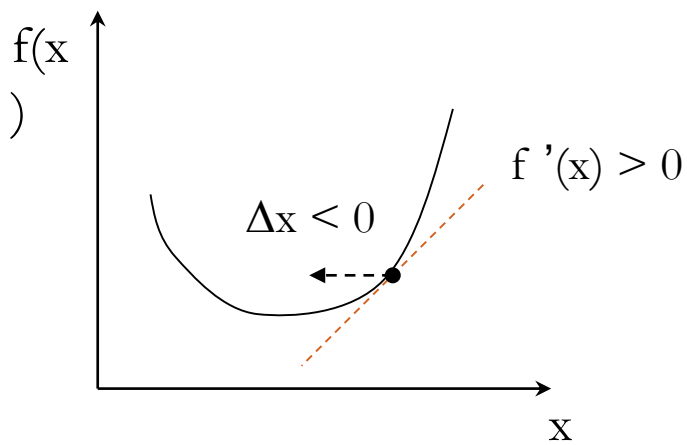
- i, j, k : neuron index for input layer (output $x_i$), hidden layer (output $y_j$), and output layer (output $y_k$) layers respectively.
- For node k in the output layer
  - the error $e_k$ is the difference between the desired output and the actual output ($e_k = d_k - y_k$).
  - The error gradient for k is: $\delta_k = y_k \cdot (1-y_k) \cdot e_k$
  - The weights are updated by: $w'_{jk} = w_{jk} + \alpha \cdot y_j \cdot \delta_k$
  - $\alpha$: learning rate (a positive number below 1)
- Similarly, for a node j in the hidden layer
  - The error gradient for k is: $\delta_j = y_j \cdot (1-y_j) \cdot \Sigma_k w_{jk} \delta_k$
  - The weights are updated by: $w'_{ij} = w_{ij} + \alpha \cdot x_i \cdot \delta_j$

# Gradient Descent

- To find minimum of f(x) from arbitrary point x
  - Local minimum can be obtained




If f '(x) < 0, increase x (dx > 0)




If f '(x) > 0, decrease x (dx < 0)

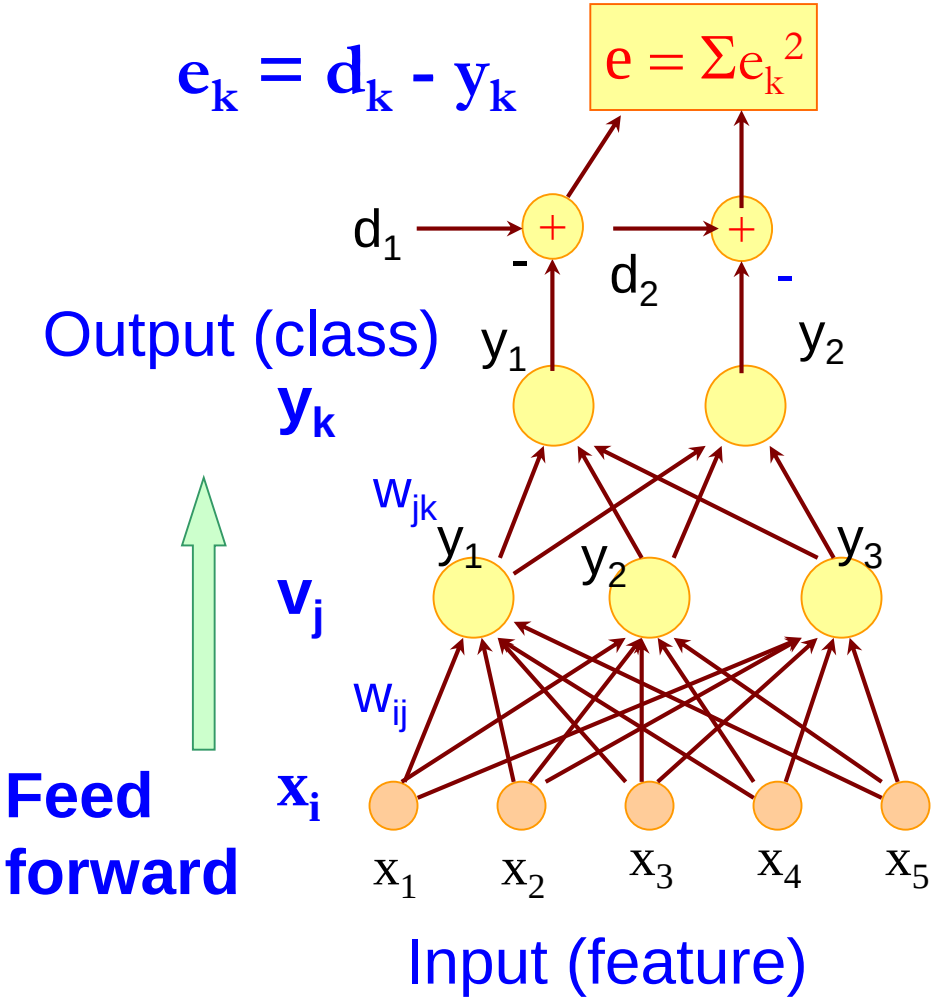
***$\Delta x = -c(df/dx)$ to find minimal f(x)***

***$x_{new} = x_{old} + \Delta x$, $\Delta x = -a(df/dx)$***

# Back Propagation (4)

$e_k = d_k - y_k$

$e = \Sigma e_k^2$




$\delta_k = g'(h_k) \cdot [d_k - y_k]$

$\Delta w_{jk} = -\eta\ (de/dw_{jk})$

$= \eta\ \delta_k\ y_j$

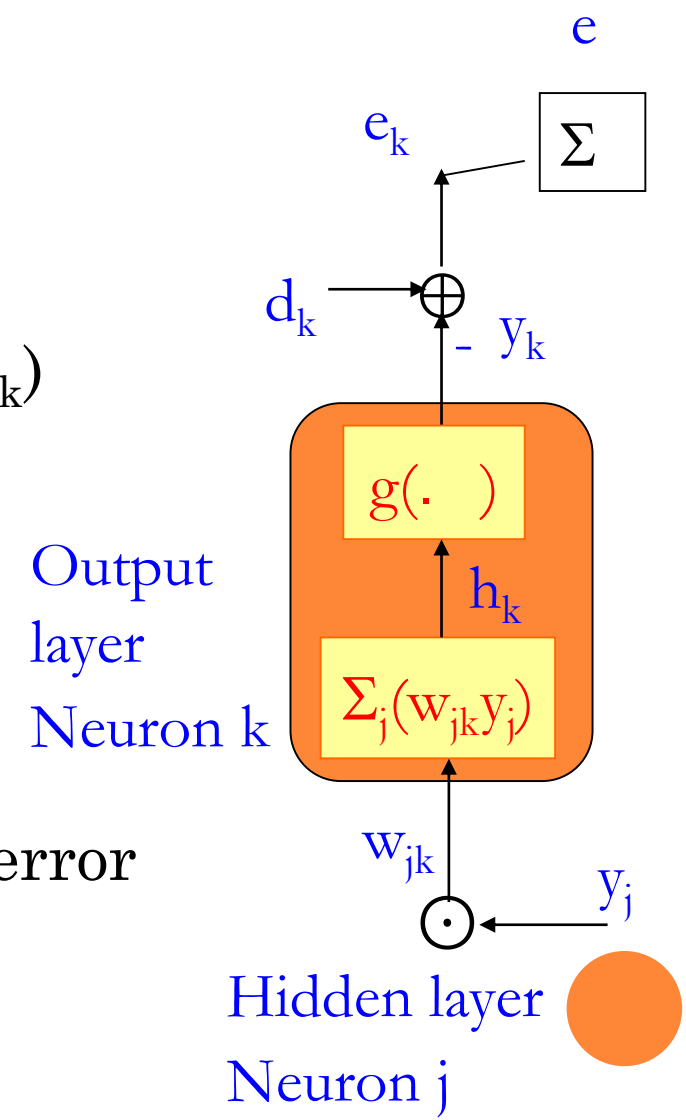
$\delta_j = g'(h_j) \cdot \Sigma_k [w_{jk} \delta_k]$

$\Delta w_{ij} = \varepsilon\ \delta_j\ x_i$

**Backward Learning**

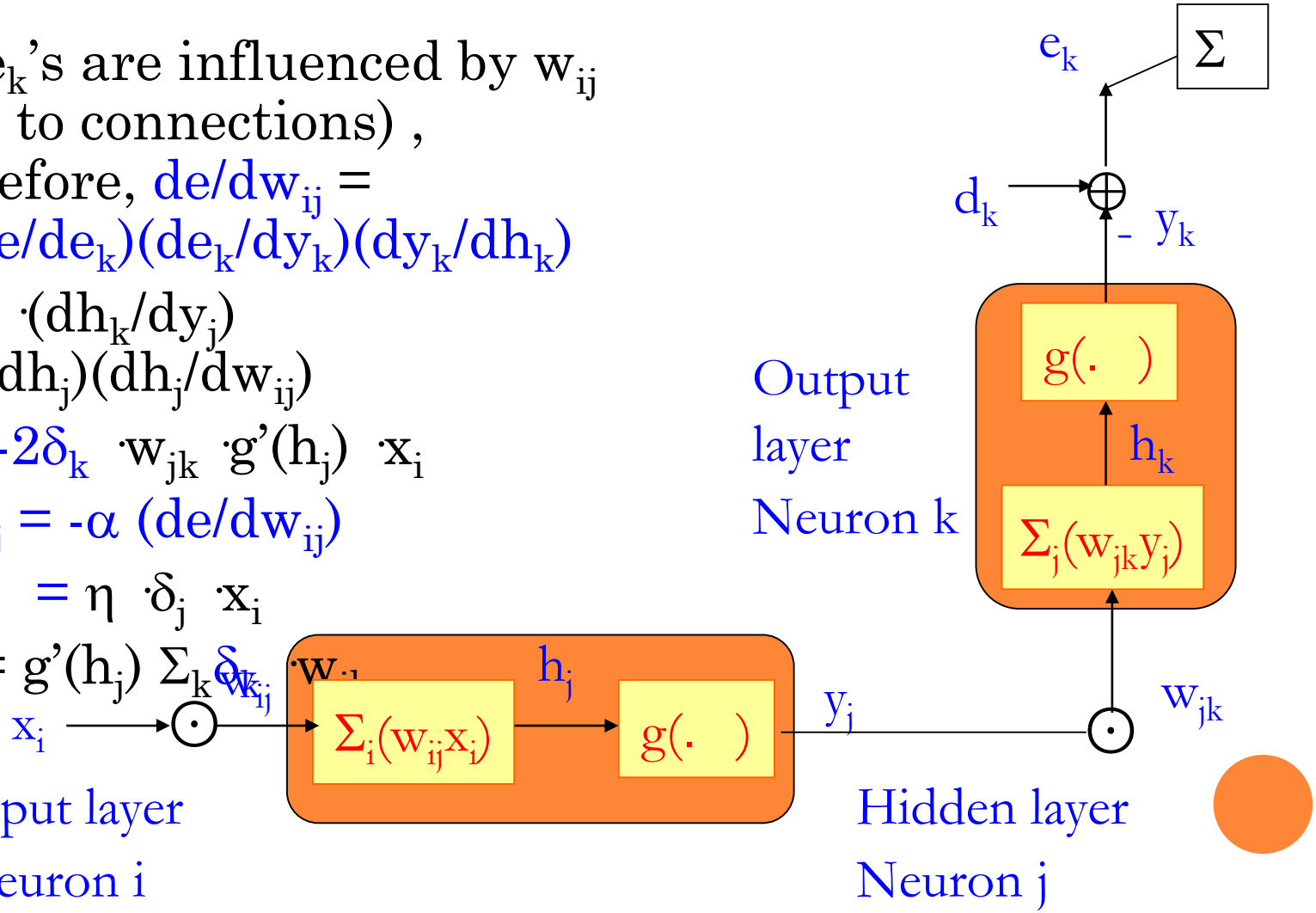
# Learning the Weights

- $e = \Sigma_k e_k^2$, $e_k = d_k - y_k$
  $y_k = g(h_k)$, $h_k = \Sigma_j w_{jk} y_j$
  by chain rule $de/dw_{jk} =$
  $(de/de_k)(de_k/dy_k)(dy_k/dh_k)(dh_k/dw_{jk})$
  $= 2e_k (-1) \cdot g'(h_k) \cdot y_j$
  $\Delta w_{jk} = -\alpha (de/dw_{jk})$
  $= -\alpha \cdot 2(d_k - y_k) (-1) \cdot g'(h_k) \cdot y_j$
  $= \eta \cdot g'(h_k) \cdot (d_k - y_k) \cdot y_j$
  $= \eta \cdot \delta_k \cdot y_j$
  $\delta_k \equiv g'(h_k) \cdot (d_k - y_k)$ feedback of error



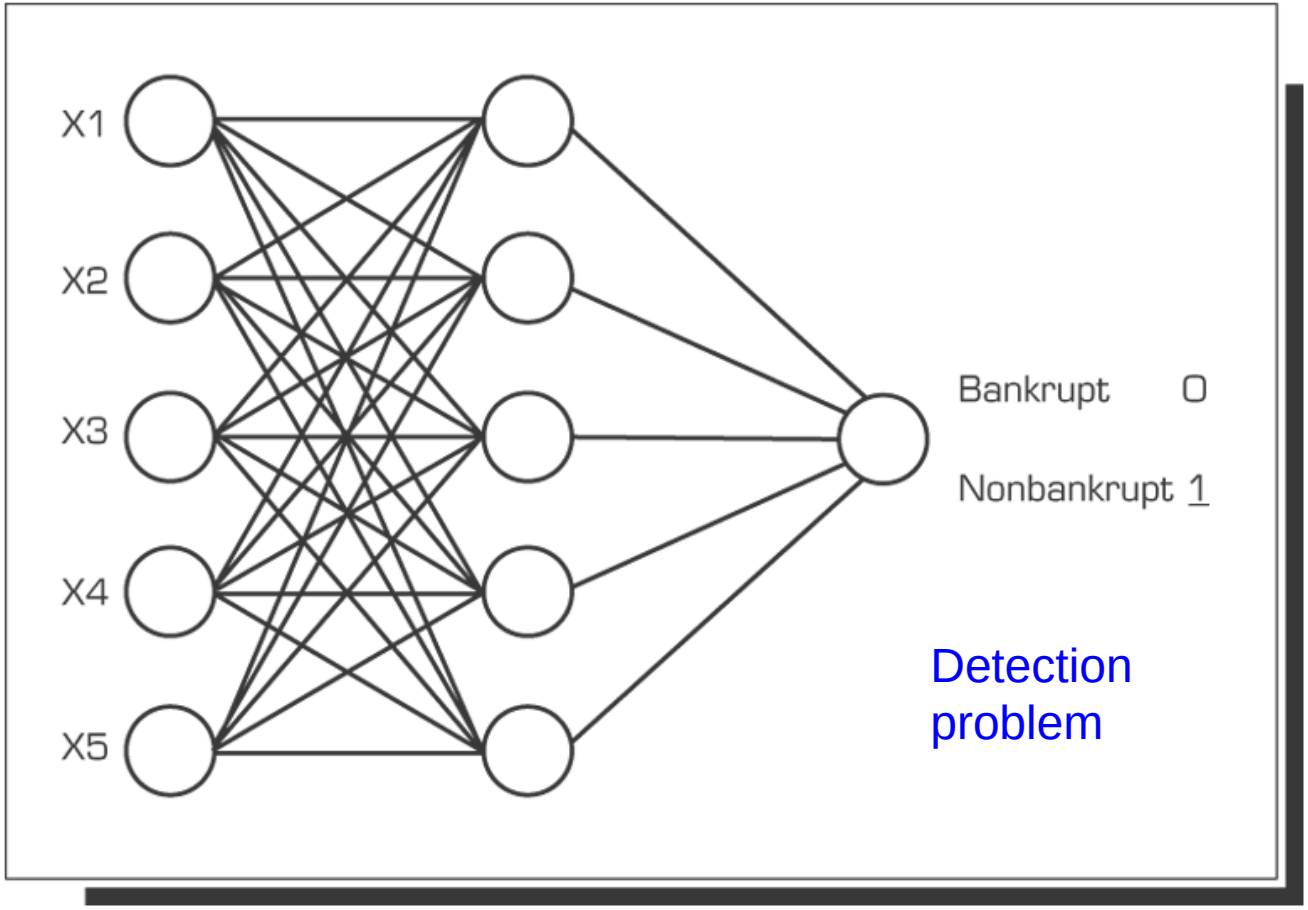
# Learning the Weights

- All $e_k$'s are influenced by $w_{ij}$ (due to connections) , therefore, $de/dw_{ij} = \Sigma_k (de/de_k)(de_k/dy_k)(dy_k/dh_k) \cdot (dh_k/dy_j)(dy_j/dh_j)(dh_j/dw_{ij})$
  $= \Sigma_k -2\delta_k \cdot w_{jk} \cdot g'(h_j) \cdot x_i$
- $\Delta w_{ij} = -\alpha\ (de/dw_{ij})$
  $= \eta \cdot \delta_j \cdot x_i$
  $\delta_j = g'(h_j)\ \Sigma_k \delta_k \cdot w_{jk}$



# Binary Classifier

**Figure 12.14** Architecture of the Bankruptcy Prediction Neural Network

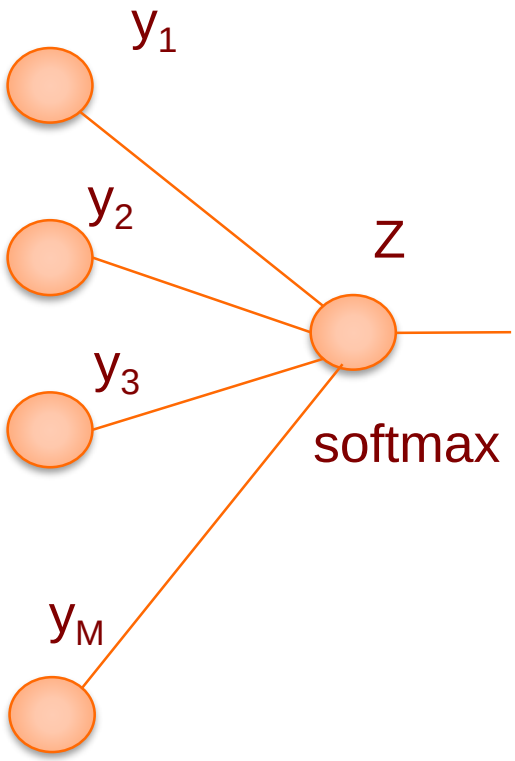



Detection problem

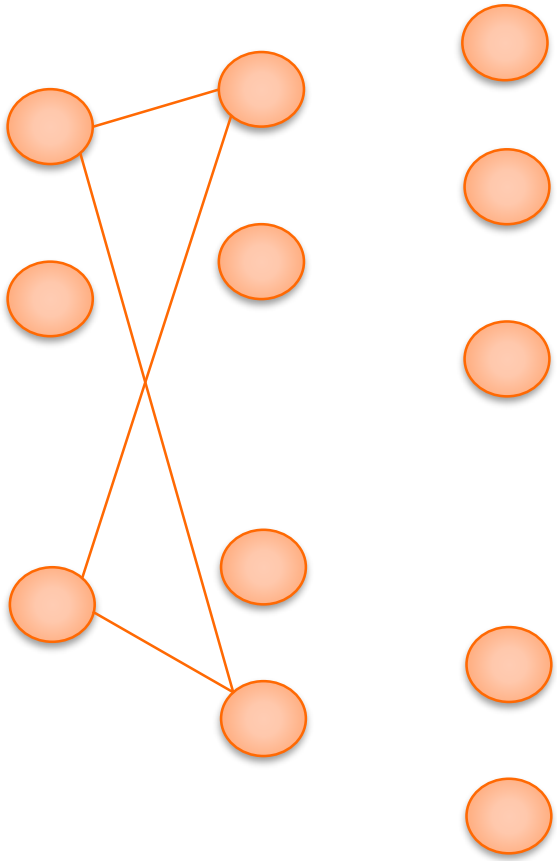
# SoftMax Node

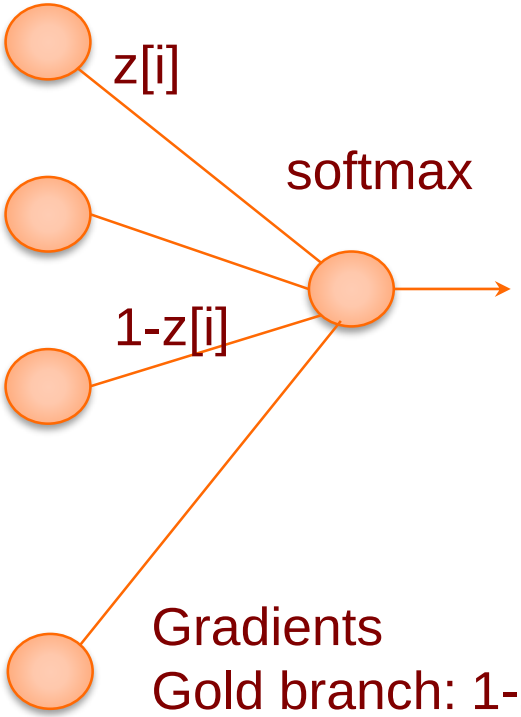
- $Z = argmax\ z_k$
- $z_k = \frac{e^{y_k}}{Y}$, $Y = \sum_{i=1}^{M} e^{y_k}$
- Think of $z_k$ as $P(C{=}C_k)$
- Objective function

$$Z = \sum_{k=1}^{M} z_k{}'\log(z_k)$$



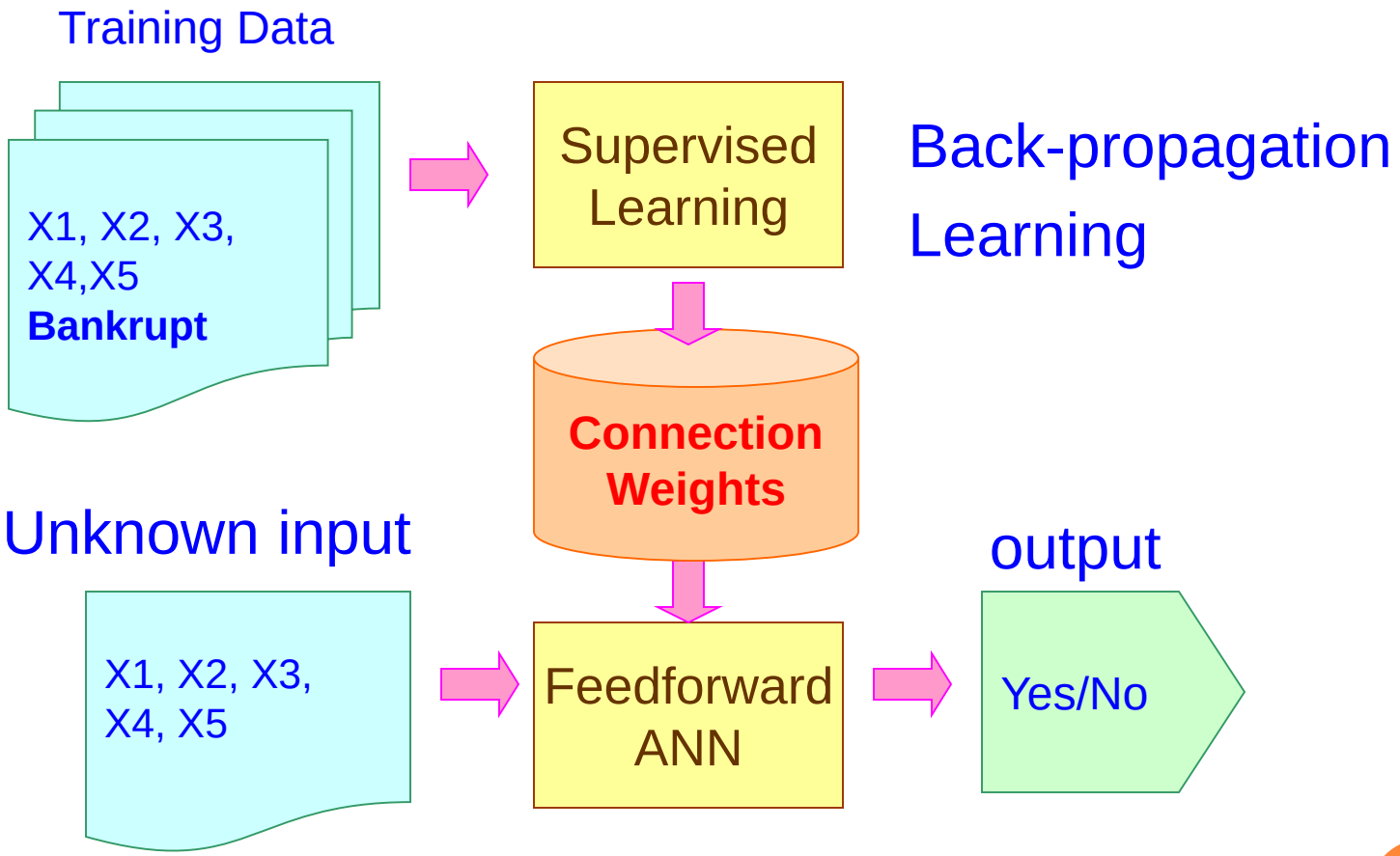
# M-ary Classifier






Gradients
Gold branch: 1-z]i]
others: -z[i]

# Decision Making With Classifier



# Improve the Speed of Convergence

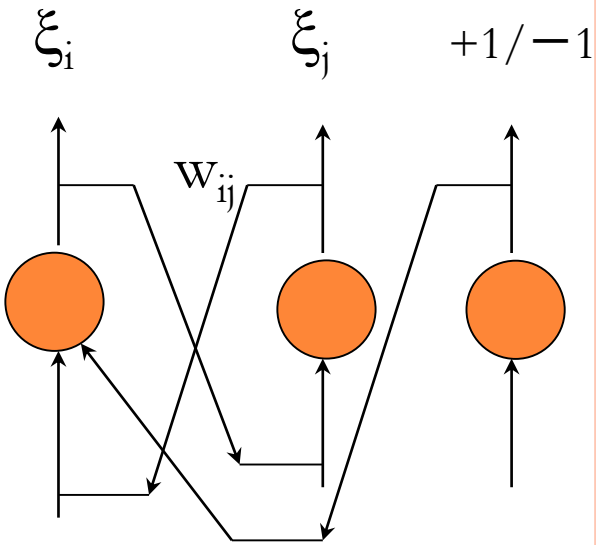
- Use generalized delta rule for learning
  - $\Delta w_{ij}(t) = \alpha \cdot x_i \cdot \delta_j + \beta \cdot \Delta w_{ij}(t-1)$
  - Momentum $\beta$ has a typical value of 0.95
  - Weighted sum between training data and original value (learn incrementally)
- Use hyperbolic tangent function instead of sigmoid
  - $\tanh(x) = 2a/(1+e^{-bx}) - a$ ($a = 1.7$, $b = 0.7$)
- Use dynamic learning rate $\alpha$
  - Increasing $\alpha$ if square-error changes in the same direction for several epochs, and decreasing $\alpha$ otherwise.

# Recurrent Networks

- Feed forward networks do not have memory.
- Recurrent networks can have connections between nodes in any layer, which enables them to store data – a memory.
- Recurrent networks can be used to solve problems where the solution depends on previous inputs as well as current inputs (e.g. predicting stock market movements).

# Hopfield Network

- $\underline{\xi} = (\xi_1, \xi_2, \ldots)$ to be memorized with values of +1/-1
- $\xi_i = \text{sign}(\Sigma_j w_{ij}\xi_j - \theta_j)$
  $w_{ij} = (1/N)\ \xi_i\ \xi_j$
- $h_i = \Sigma_j w_{ij}\xi_j = (1/N)\Sigma_j \xi_i \xi_j \xi_j$
  $= \xi_i$ (since $\xi_j^2 = 1$)
- $h_i = \Sigma_j w_{ij} S_j = (1/N)\Sigma_j \xi_i \xi_j S_j$
  $=(1/N)\ \xi_i(\Sigma_j \xi_j S_j)$
  $= (1/N)\ \xi_i \cdot \langle \underline{S}, \underline{\xi} \rangle$




$\underline{S} = (S_1, S_2, \ldots, S_n)$

If $\underline{S}$ is close to $\underline{\xi}$ (in Hamming distance) such that $\langle \underline{S}, \underline{\xi} \rangle > 0$, then $S_i$ will be attracted into $\xi_i$ and ($\underline{S} = \underline{\xi}$)

# Hopfield Network (cont'd)

$$W = \sum_{i=1}^{N} X_i X_i^t - N\,I$$

$$Y = sign(WX - \theta)$$

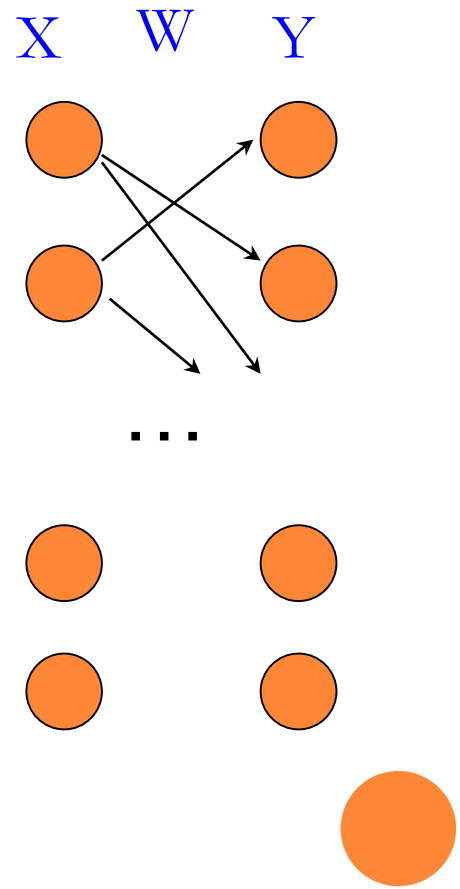
$X_i$'s are patterns to be memorized

Weights:N patterns ($X_i$'s)

$Y_i = \text{Sign}(WX_i - \theta) = X_i$

Input X close to $X_i$ will be attracted to $X_i$

Hopfield network is a memory that usually maps an input vector to the memorized vector whose **Hamming distance** from the input vector is least. (auto-associative memory) (memory→recall)



# Example of Hopfield Network

$$X_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \quad X_2 = \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \end{bmatrix} \quad X_3 = \begin{bmatrix} 1 \\ -1 \\ 1 \\ 1 \\ -1 \end{bmatrix}$$

$$W = \sum_{i=1}^{3} X_i X_i^t - 3I = \begin{bmatrix} 0 & 1 & 3 & 3 & 1 \\ 1 & 0 & 1 & 1 & 3 \\ 3 & 1 & 0 & 3 & 1 \\ 3 & 1 & 3 & 0 & 1 \\ 1 & 3 & 1 & 1 & 0 \end{bmatrix}$$

$$Y_1 = sign(WX_1) = sign(\begin{bmatrix} 8 \\ 6 \\ 8 \\ 8 \\ 6 \end{bmatrix}) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = X_1$$

$$Y_2 = WX_2 = X_2 \qquad Y_3 = WX_3 = X_3$$

$$X_4 = \begin{bmatrix} 1 \\ 1 \\ -1 \\ 1 \\ 1 \end{bmatrix}$$

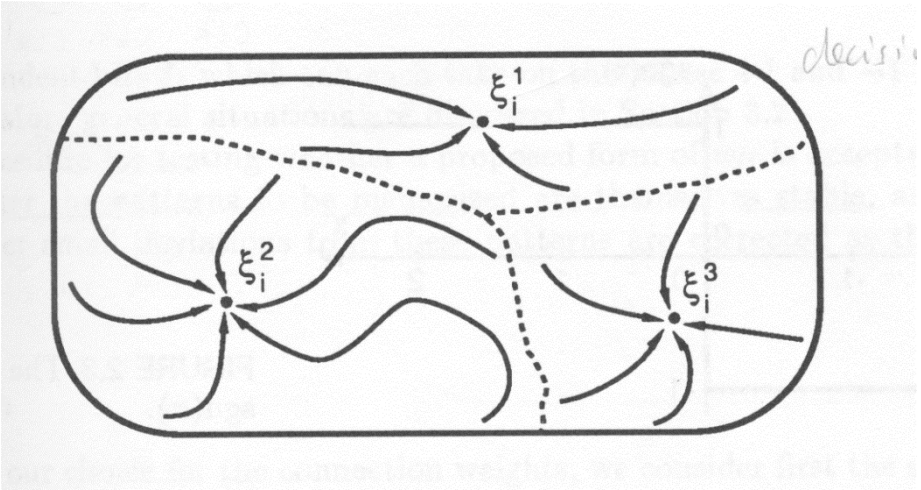
$$Y_4 = sign(WX_4) = sign(\begin{bmatrix} 2 \\ 4 \\ 8 \\ 2 \\ 4 \end{bmatrix}) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = X_1$$

# Example of Hopfield Network (cont'd)

$$X_5 = \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \\ 1 \end{bmatrix}$$

$$Y_5 = sign(WX_5) = sign(\begin{bmatrix} 2 \\ 2 \\ 2 \\ -4 \\ 2 \end{bmatrix}) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \\ 1 \end{bmatrix}$$

$$Y_5' = WY_5 = sign(\begin{bmatrix} 2 \\ 4 \\ 2 \\ 8 \\ 4 \end{bmatrix}) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = X_1$$




Auto-associative Memory

Storage: setting the weights

Retrieval: retrieving data

# Application of Hopfield Network

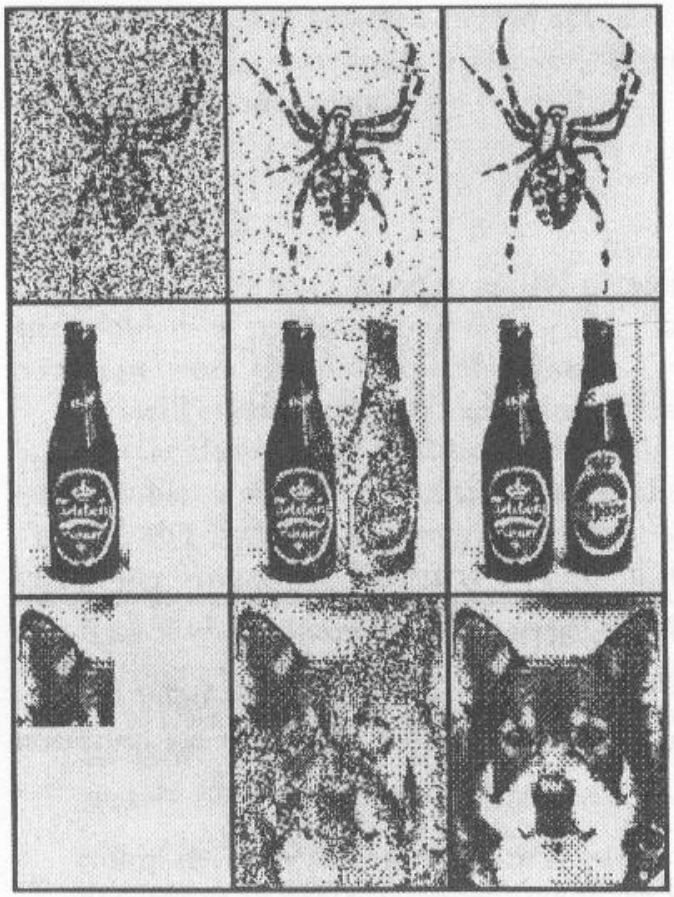


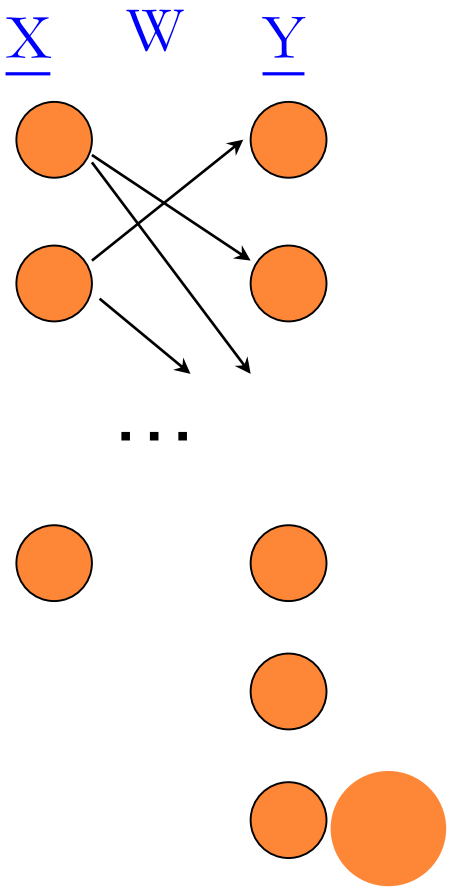
FIGURE 2.1 Example of how an associative memory can reconstruct images. These are binary images with 130 × 180 pixels. The images on the right were recalled by the memory after presentation of the corrupted images shown on the left. The middle column shows some intermediate states. A sparsely connected Hopfield network with seven stored images was used.

# Bidirectional Associative Memories

$$W = \sum_{i-1}^{n} X_i Y_i^t$$

$$sign(W^t X_j) = Y_j$$

$$sign(W Y_j) = X_j$$




- association X → Y

# Example of BAM

$$X_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad X_2 = \begin{bmatrix} -1 \\ -1 \end{bmatrix} Y_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad Y_2 = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}$$

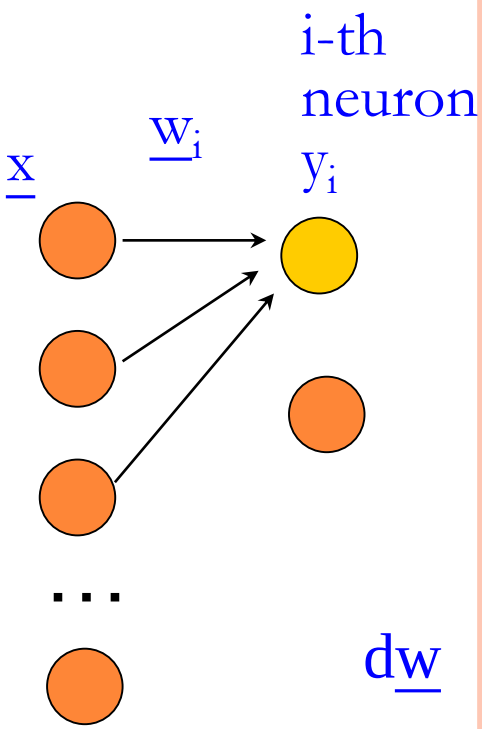
$$W = \sum_{i=1}^{2} X_i Y_i^t = \begin{bmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix}$$

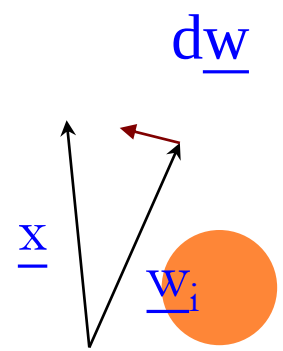
$$sign(W^t X_1) = sign(\begin{bmatrix} 4 \\ 4 \\ 4 \end{bmatrix}) = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = Y_1$$

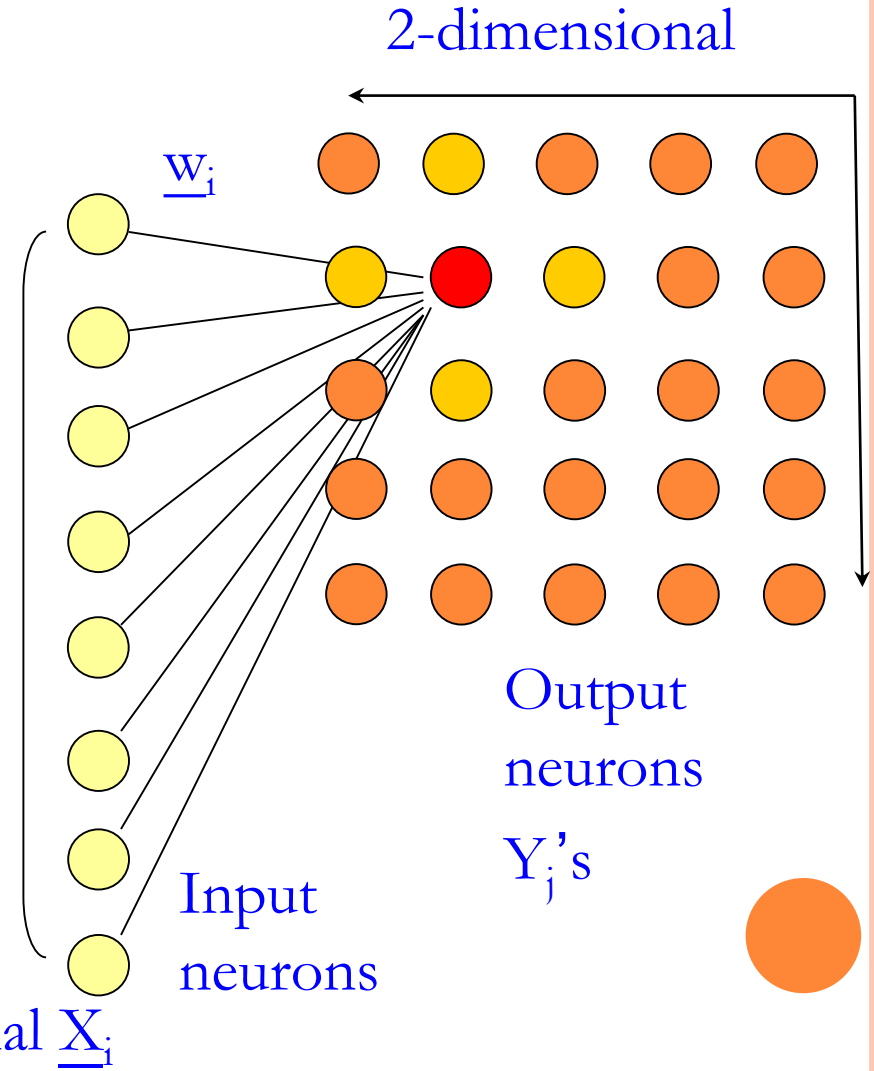
$$sign(WY_1) = sign(\begin{bmatrix} 6 \\ 6 \end{bmatrix}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix} = X_1$$

# Kohonen Maps (Self-Organizing Map)

- Unsupervised learning
  - Clustering for training data $\{\underline{x}_j\}$
- Winner-take-all
  - The winner is the neuron with highest output ($\underline{w}_i$ closest to $\underline{x}$)
  - The winner and its spatial neighborhoods can update weights
  - $w_{ij}' = w_{ij} + \alpha(x_j - w_{ij})$ or $d\underline{w}_i = \alpha(\underline{x} - \underline{w}_i)$
- $y_i = \text{sign}(<\underline{x}, \underline{w}_i>)$
  - $<\underline{x}, \underline{w}_i>$ similarity
- 和KNN相似, K個output neurons的weights代表K個centroids



# Kohonen Maps (cont'd)

- N-dimensional → 2-D
  - 2D Visual display for closeness for N-D space
- Codewords ($\underline{w}_i$) close to one another in n-dimensional space will be close on 2-D plane
- The weights of farther neurons are less modified
- View N-D space on 2D plane